



Towards Lossless Memory-efficient Training of Spiking Neural Networks via Gradient Checkpointing and Spike Compression

Yifan Huang^{1,2,3}, Wei Fang⁴, Zecheng Hao^{1,2,5}, Zhengyu Ma³, Yonghong Tian^{1,2,3,4}

¹School of Computer Science, Peking University ²Beijing Key Laboratory of Brain-inspired Spiking Large Models ³Peng Cheng Laboratory

⁴School of Electronic and Computer Engineering, Shenzhen Graduate School, Peking University ⁵State Key Laboratory for Multimedia Information Processing



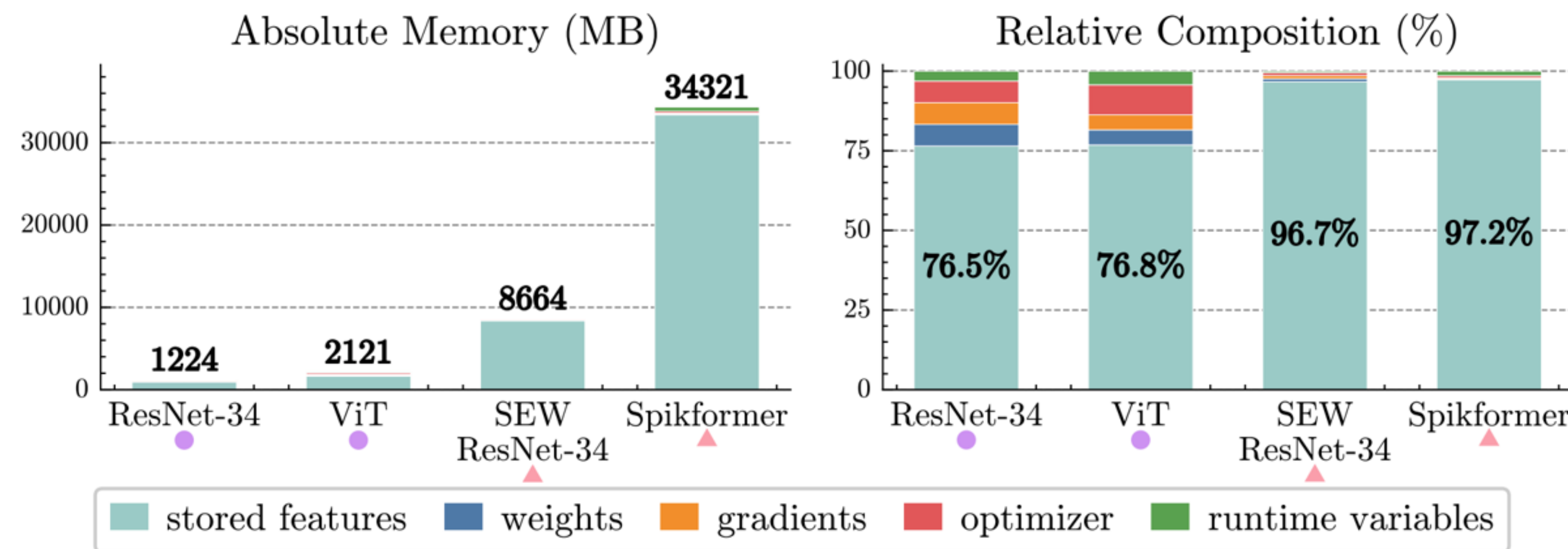
BPTT Training of SNNs Consumes Massive GPU Memory

Spiking neural networks (SNNs) process information through spike dynamics over time. To train a SNN directly, the temporal dimension is first discretized into T time steps so the network can be viewed as an RNN-like model. Then, the SNN is optimized with **backpropagation through time (BPTT)**.

BPTT incurs substantial memory overhead.

- requires much more GPU memory than ANN training
- memory cost is dominated by **stored internal features**

Reducing internal feature storage is therefore the key priority.



Key considerations:

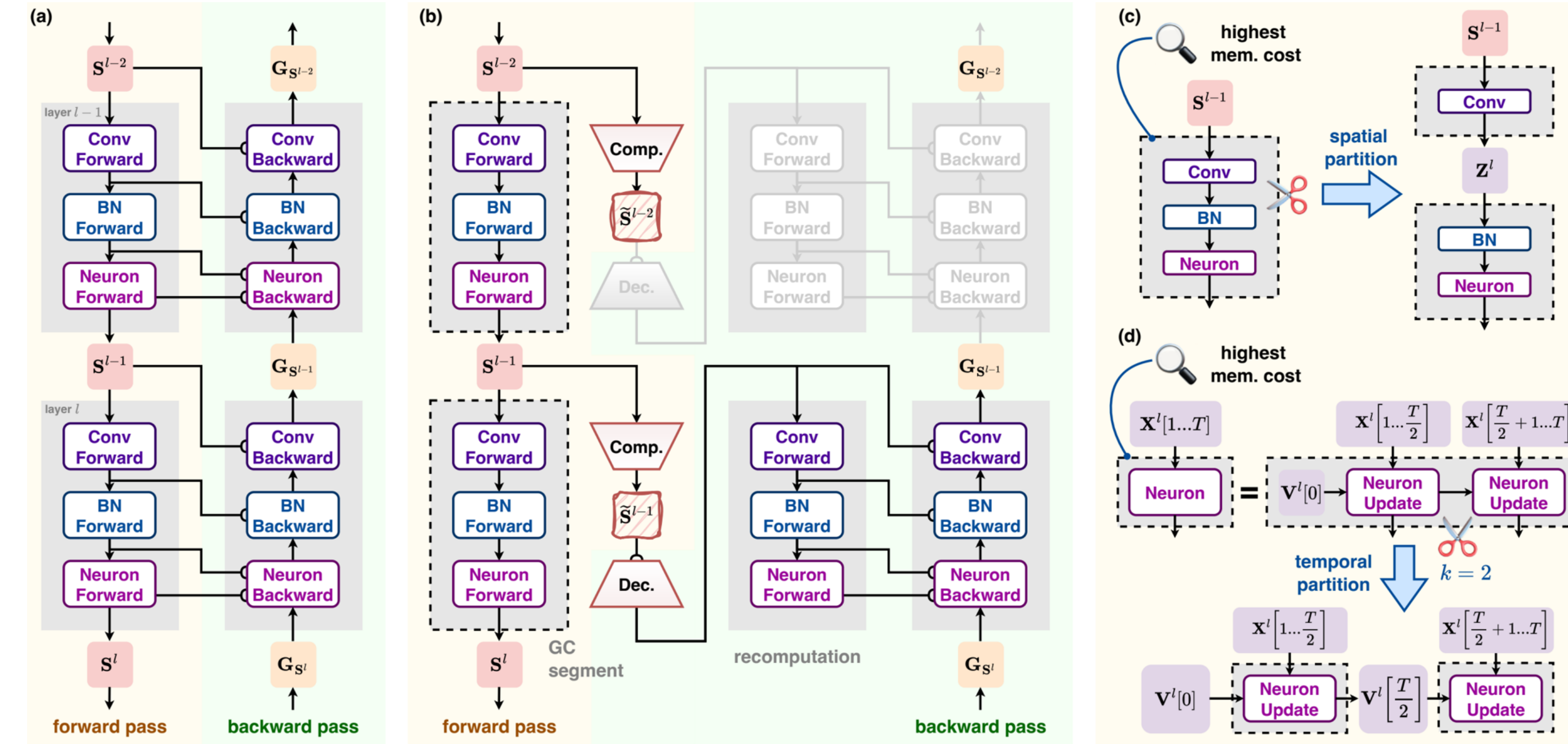
- reduce memory **without sacrificing accuracy or speed**
- maintain **generality** across different SNN architectures and tasks
- remain **easy to use**

Contribution

We propose an **automatic memory optimization pipeline** for SNN training that is broadly applicable, easy to use, and preserves training speed and model performance. The API and tutorials are available in **SpikingJelly**.

```
from spikingjelly.activation_based import memopt
net = memopt.memory_optimization(
    net, instance=(VGGBlock,),
    dummy_input=(torch.zeros(32, 3, 224, 224),),
    compress_x=True, level=4, temporal_split_factor=2,
)
```

Method

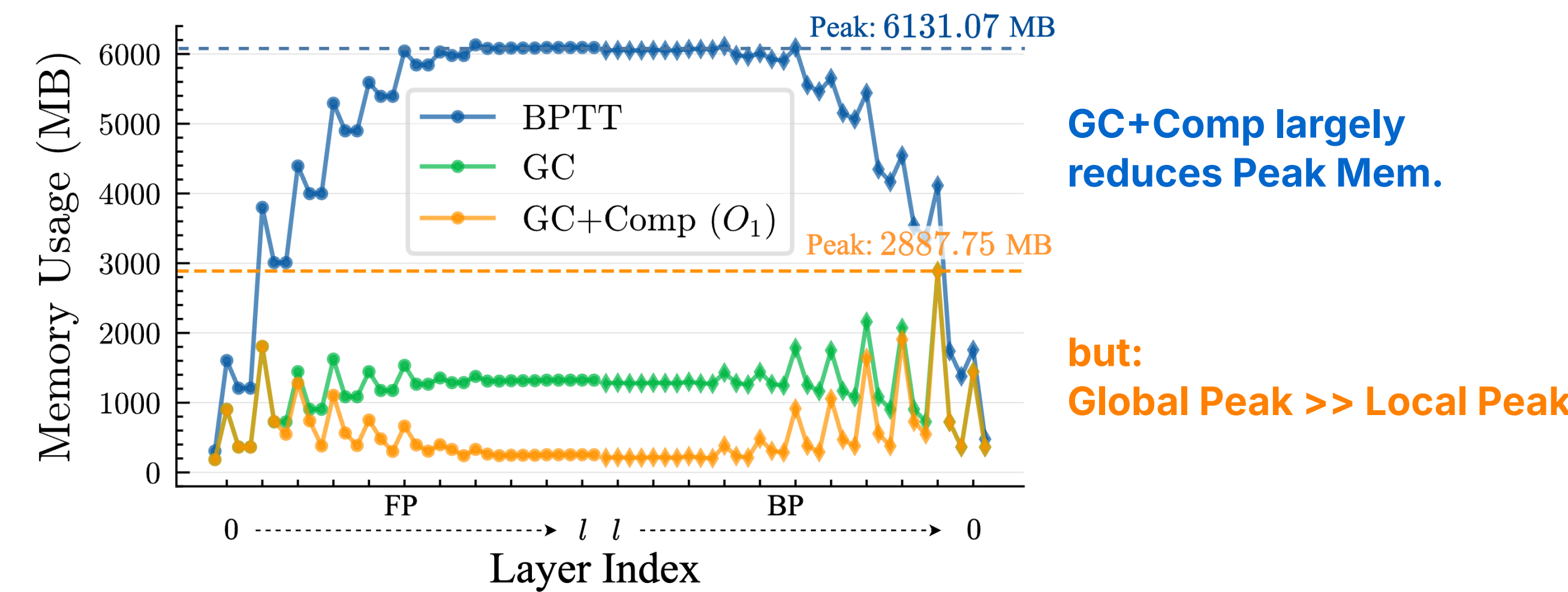


O₁: Layer-wise gradient checkpointing (GC)

- save only the spike inputs for each layer; discard the other features
- reconstruct the required intermediate features during the backward pass
- each checkpointed layer is called a **GC segment**

O₂: Lossless spike compression

- bit-pack** spike tensors (formerly in FP32 or FP16)
- decompress them before local recomputation begins



O_{3,4}: Adaptive adjustment

- spatio-temporal partitioning**: split the critical segment across layers or time steps to reduce peak memory
- greedy restoration**: revert high-time-cost GC segments to standard BPTT blocks, avoiding unnecessary recomputation and speeding up training

Results

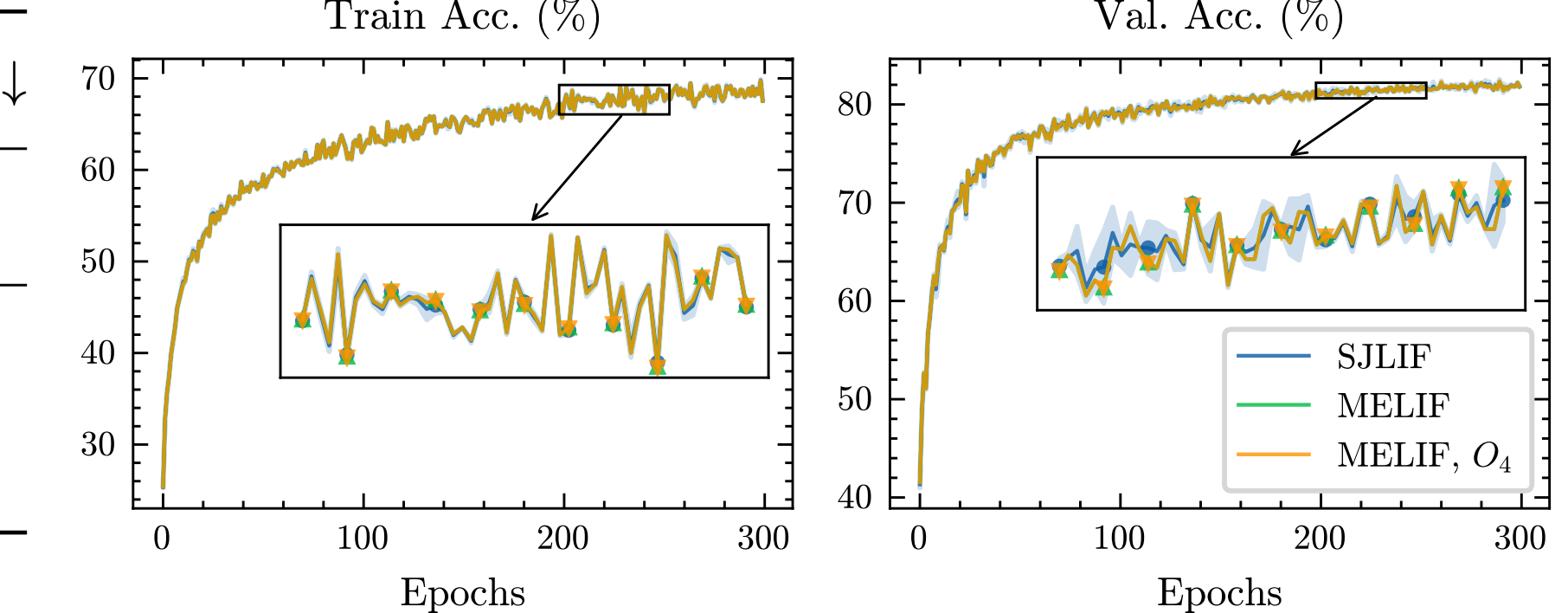
Up to $8\times$ memory efficiency with $\leq 20\%$ training slowdown

Task	T	Batch Size	Network	LIF impl.	Method	Throughput (sample / s) $\uparrow$	Peak Alloc. Mem. (MB) $\downarrow$
Sequential CIFAR-10	32	128	SCNN	SJLIF	BPTT	4872.23	1317.23
				PTLIF	BPTT	1054.68	1264.97
				MELIF	O ₄	5138.76 (1.05 $\times$)	474.98 (0.36 $\times$)
DVS128 Gesture	16	16	7B-Net	SJLIF	BPTT	114.52	8984.02
				PTLIF	BPTT	36.52	8067.41
				MELIF	O ₄	120.04 (1.05 $\times$)	4213.86 (0.47 $\times$)
CIFAR10-DVS	10	32	Spiking VGG	SJLIF	BPTT	290.26	6131.07
				PTLIF	BPTT	150.69	5889.44
				MELIF	O ₄	270.79 (0.93 $\times$)	2349.39 (0.38 $\times$)
			SEW ResNet-34	SJLIF	BPTT	309.04	8821.28
				PTLIF	BPTT	202.83	7140.09
				MELIF	O ₄	281.39 (0.91 $\times$)	2004.14 (0.23 $\times$)
ImageNet	4	32	Spikformer (8-512)	SJLIF	BPTT	116.70	34264.76
				PTLIF	BPTT	71.03	28779.13
				MELIF	O ₄	93.58 (0.80 $\times$)	7640.68 (0.22 $\times$)
			QKFormer (10-512)	SJLIF	BPTT	86.15	44571.33
				PTLIF	BPTT	55.65	37375.90
				MELIF	O ₄	76.51 (0.89 $\times$)	5219.93 (0.12 $\times$)

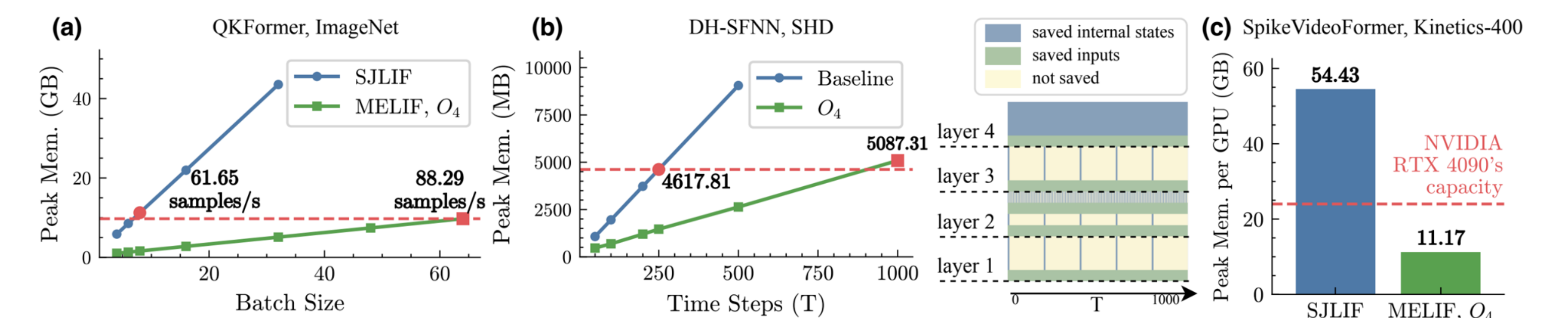
Ablation study

LIF impl.	Opt. Level	Throughput (sample / s) $\uparrow$	Peak Alloc. Mem. (MB) $\downarrow$
SJLIF	—	290.26	6131.07
PTLIF	—	150.69	5889.44
MELIF	—	331.30	4865.06
	O ₁	246.81	2887.75
	O ₃	247.83	2349.39
	O ₄	270.79	2349.39

No accuracy loss



Enables larger batch sizes, more time steps, and lower hardware requirements



An extension to the **SpikingJelly** ecosystem.

Project: <https://github.com/AllenYolk/snn-gradient-checkpointing>
SpikingJelly: <https://github.com/fangwei123456/spikingjelly>